

CS4421: DevOps

Week 4: Cloud Providers, Infrastructure as Code (IaC) & Edge Release

Instructor: Robert Bordianu

Module: CS4421 - DevOps (8-Week Curriculum)

Primary Stack: Astro (SSG), AWS S3, AWS CloudFront, AWS CDK (TypeScript), GitHub Actions

Today's Agenda (3 x 2-Hour Blocks)

- **Block 1 (Quiz + Cloud Architectures):**
 - 🕒 **Quiz 3 (First 15 mins — Locked Single Tab):** Covers Week 3
 - Cloud Computing Models & Providers Landscape: **AWS vs. GCP vs. Azure vs. Vercel/Cloudflare**
 - Static Site Generation (SSG) Architecture & Immutable Build Artifacts (`dist/`)
 - Object Storage (S3) vs. Edge CDNs (CloudFront)
- **Block 2 (Tooling & Architecture):**
 - **Infrastructure as Code (IaC):** Why ClickOps Fails; Declarative vs. Imperative
 - IaC Tooling Comparison: **Terraform vs. CloudFormation vs. AWS CDK vs. Pulumi**
 - **AWS CDK in TypeScript:** Declaring S3, CloudFront & Auto-Invalidation
 - **Sample Pipeline:** `deploy-static.yml` with AWS CDK
 - Edge Caching, Invalidation & The `git revert` Rollback Protocol
- **Block 3 (Hands-on Lab):**
 - Automated S3 + CloudFront Deployment via AWS CDK in GitHub Actions
 - **Hands-on Rollback Drill 1:** Fast Recovery via `git revert`

BLOCK 1: Quiz 3 & Theory

Cloud Architectures, Provider Landscape & Edge Distribution

Quiz 3 Guidelines (15 Minutes)

- **Scope:** Week 3 QA, Testing Pyramid, Shift-Left Security, CI Tooling (GitHub vs. Jenkins), Actions YAML.
- **Integrity Rules:**
 - Strict single-tab lockdown mode via Brightspace.
 - No external tabs, no terminal access, no AI tools.
- **Question Format:**
 - 10 Questions (5 options each, 1 pt each), embedded YAML workflow failure logs, Vitest assertion traces, and multiple-select questions.
- *When the timer expires, the quiz automatically submits.*

Cloud Providers & Hosting Landscape

Category	Platforms	Key Characteristics	Best Suited For
Hyperscalers (IaaS/PaaS)	AWS, Google Cloud (GCP), Microsoft Azure	Complete infrastructure catalog (compute, storage, IAM, networks)	Enterprise apps, containers, scalable cloud backends
Developer Edge Platforms (PaaS)	Vercel, Netlify, Cloudflare Pages	Git-integrated zero-config deployments, edge functions	Fast static frontend & Jamstack deployments
Bare Metal / VPS	DigitalOcean, Hetzner, Linode	Low cost, straightforward virtual servers	Simple apps, self-hosted databases

Our Strategy: We use **AWS** to master foundational cloud primitives (S3, CloudFront, ECR, IAM) that transfer across all cloud providers.

Source Code vs. Build Artifacts



- **Deterministic Builds:** The exact same commit hash must produce the exact same byte-for-byte output in `dist/`.
- **Security & Performance:** HTML/CSS/JS files contain zero server dependencies, zero open database ports, and lightning-fast delivery.

Cloud Infrastructure: S3 vs. CloudFront



1. Amazon S3 (Object Storage)

- Highly durable (99.999999999%).
- Stores files (blobs), not computation.
- Holds canonical static build artifacts.

2. AWS CloudFront (CDN)

- 400+ Points of Presence (PoPs) globally.
- Low latency (< 15ms) edge caching.
- Free automated SSL/TLS certificates.

BLOCK 2: Tooling & Architecture

Infrastructure as Code (IaC) & AWS CDK

Infrastructure as Code (IaC): Why ClickOps Fails

- **The Anti-Pattern: ClickOps:** Manually clicking buttons in the AWS web console.
 - ✗ Non-reproducible (cannot recreate staging or disaster recovery environments).
 - ✗ **Configuration Drift:** Dev, Staging, and Prod slowly become out of sync.
 - ✗ Zero version control, audit trail, or automated review for infrastructure.

ClickOps (Manual Web UI)

- Click "Create Bucket"
- Click "Allow Public"
- Hope nobody misclicks

VS.

Infrastructure as Code (Declarative Files)

```
const bucket =  
  new s3.Bucket(...);
```

—> Version Controlled,
Audited & Reusable

Infrastructure as Code (IaC) Tooling Landscape

Tool	Paradigm	Language	Best Suited For
Terraform / OpenTofu	Declarative	HCL (HashiCorp Config Language)	Multi-cloud infrastructure orchestration (Industry Standard)
AWS CloudFormation	Declarative	JSON / YAML	Native AWS stack templates and automated provisioning
AWS CDK	Imperative / Object-Oriented	TypeScript, Python, Go	Defining AWS cloud resources using real programming languages
Pulumi	Imperative / Declarative	TypeScript, Python, Go, C#	Multi-cloud infrastructure defined via modern programming stacks

Key Takeaway: In modern DevOps, infrastructure is written, peer-reviewed, and deployed just like software!

AWS CDK in Action: Declaring S3 + CloudFront

With **AWS Cloud Development Kit (CDK)** in TypeScript, our infrastructure is code in `cdk/lib/static-site-stack.ts` :

```
import * as cdk from 'aws-cdk-lib';
import * as s3 from 'aws-cdk-lib/aws-s3';
import * as cloudfront from 'aws-cdk-lib/aws-cloudfront';
import * as origins from 'aws-cdk-lib/aws-cloudfront-origins';
import * as s3deploy from 'aws-cdk-lib/aws-s3-deployment';

export class StaticSiteStack extends cdk.Stack {
  constructor(scope: cdk.App, id: string, props?: cdk.StackProps) {
    super(scope, id, props);

    const siteBucket = new s3.Bucket(this, 'SiteBucket', {
      removalPolicy: cdk.RemovalPolicy.DESTROY,
      autoDeleteObjects: true,
    });

    const distribution = new cloudfront.Distribution(this, 'SiteDistribution', {
      defaultBehavior: { origin: origins.S3BucketOrigin.withOriginAccessControl(siteBucket) },
      defaultRootObject: 'index.html',
    });

    new s3deploy.BucketDeployment(this, 'DeploySite', {
      sources: [s3deploy.Source.asset('./dist')],
      destinationBucket: siteBucket,
      distribution,
      distributionPaths: ['/*'], // Automatic CloudFront CDN cache invalidation!
    });
  }
}
```

Sample Pipeline: `deploy-static.yml` (AWS CDK)

```
name: Deploy Static Site via AWS CDK
on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Source Code
        uses: actions/checkout@v4

      - name: Setup Node.js Runtime
        uses: actions/setup-node@v4
        with:
          node-version: 24
          cache: 'npm'

      - name: Install Project & CDK Dependencies
        run: npm ci

      - name: Build Astro Static Site
        run: npm run build

      - name: Configure AWS Credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: us-east-1

      - name: Deploy Infrastructure & Assets with AWS CDK
        run: npx cdk deploy --require-approval never
```

The Problem of Stale Assets: Cache Invalidation

- When CloudFront caches `index.html` at the edge, it serves that copy for 24 hours by default.
- If you push a bug fix to S3, users will **still see the broken version** until the edge cache is invalidated.

Manual CLI Invalidation:

```
aws cloudfront create-invalidation \  
  --distribution-id $DIST_ID \  
  --paths "/*"
```

Automated via AWS CDK:

```
new s3deploy.BucketDeployment(this, 'DeploySite', {  
  sources: [s3deploy.Source.asset('./dist')],  
  destinationBucket: siteBucket,  
  distribution,  
  distributionPaths: ['/*'], // Purges edge globally!  
});
```

- CloudFront purges old edge copies worldwide in < 30 seconds.

Emergency Rollback Under Branch Protection Rules

When `main` has **Branch Protection Rules** enabled (*Require a pull request before merging*):

✗ You **cannot** run `git push origin main` directly (GitHub blocks the push with error `GH006`).

Option A: GitHub UI (Fastest)

1. Open the merged broken PR on GitHub.
2. Click the "**Revert**" button.
3. GitHub automatically creates a `revert-12-feat` branch and opens a Revert PR.
4. Approve & merge into `main`.

Option B: Terminal Git Flow

```
# 1. Create emergency rollback branch
git checkout -b revert/fix-outage

# 2. Invert the bad commit
git revert HEAD --no-edit

# 3. Push and open Revert PR
git push -u origin revert/fix-outage
# Merge PR into main -> CI deploys!
```

Bad PR Merged → Open Revert PR → Merge into main → Automated CDK Deploy & Edge Invalidation (< 2 mins)

BLOCK 3: Hands-on Practical Lab

Automated S3/CloudFront Deployment via AWS CDK

Lab 4 Objectives (Individual Repos)

1. Scaffold an AWS CDK TypeScript app inside your repository (`mkdir cdk && cd cdk && npx cdk init app --language typescript`).
2. Declare your Amazon S3 bucket, AWS CloudFront distribution, and `BucketDeployment` in `cdk/lib/static-site-stack.ts` .
3. Configure encrypted AWS credentials in GitHub Repository Secrets (`AWS_ACCESS_KEY_ID` , `AWS_SECRET_ACCESS_KEY`).
4. Author `.github/workflows/deploy-static.yml` to build Astro and run `npx cdk deploy --require-approval never` on push to `main` .
5. **Execute Hands-on Rollback Drill 1 (Under Branch Protection):**
 - Open a feature PR with an intentional breaking UI bug and merge to `main` .
 - Observe live broken production at your CloudFront URL.
 - Open and merge an **Emergency Revert PR**; watch automated CDK recovery and CDN cache purge in < 2 minutes.

Step-by-Step Lab Commands

```
# 1. Build Astro static site locally & synthesize CDK CloudFormation template
npm run build # Generates dist/
cd cdk && npx cdk synth && cd ..

# 2. Add deploy-static.yml to .github/workflows/ and push on a feature branch
git checkout -b feat/add-cdk-pipeline
git add .github/workflows/deploy-static.yml cdk/
git commit -m "feat(cd): configure automated S3 and CloudFront deployment via AWS CDK"
git push -u origin feat/add-cdk-pipeline

# 3. Open PR -> Review -> Merge into main -> Watch GitHub Actions CDK deploy!
# 4. Open your live CloudFront HTTPS domain in the browser!

# 5. ROLLBACK DRILL: Merge a broken PR -> open Revert PR -> Merge -> Verify recovery!
```

Week 4 Deliverables & Checkpoint

- [] Live, publicly accessible HTTPS URL on AWS CloudFront.
- [] AWS CDK stack (`cdk/lib/static-site-stack.ts`) committed to repository.
- [] `.github/workflows/deploy-static.yml` automatically deploying every merge to `main` via `npx cdk deploy`.
- [] Automated CloudFront cache invalidation working through `BucketDeployment`.
- [] **Rollback Drill 1 completed under Branch Protection:** Revert PR merged and verified in < 2 minutes.

⚠ Reminder for Next Week:

Quiz 4 takes place in the **first 15 minutes of Week 5 Block 1** (Locked Single Tab).

Covers Week 4 Cloud Providers, IaC Fundamentals, AWS CDK, S3, CloudFront, Caching, and Protected Branch Rollbacks.